AI Assisted Coding

Assignment . 13.5

Name : K.Laxman

Ht.No : 2303A51428

Batch : 21

Task Description #1 (Refactoring – Removing Global Variables)

• Task: Use AI to eliminate unnecessary global variables from the code.

• Instructions:

o Identify global variables used across functions.

o Refactor the code to pass values using function parameters.

o Improve modularity and testability.

• Sample Legacy Code:

```
rate = 0.1
def calculate_interest(amount):
return amount * rate
print(calculate_interest(1000))
```

• Expected Output:

o Refactored version passing rate as a parameter or using a configuration structure.

Code :

```
'''
Task Description #1 (Refactoring – Removing Global Variables)
• Task: Use AI to eliminate unnecessary global variables from the
code.
• Instructions:
o Identify global variables used across functions.
o Refactor the code to pass values using function parameters.
o Improve modularity and testability.
• Sample Legacy Code:
rate = 0.1
def calculate_interest(amount):
return amount * rate
print(calculate_interest(1000))
• Expected Output:
o Refactored version passing rate as a parameter or using a
configuration structure.
'''
# Refactored Code without Global Variables
def calculate_interest(amount, rate):
    return amount * rate
# Example usage
interest_rate = 0.1
print(calculate_interest(12000, interest_rate))
```

Output:

```
C:\Users\acera\Desktop\Btech_3_2\AI Assistant coding>python
1200.0
```

Observation:

Removing Global Variables

AI identified the global rate and rewrote calculate_interest to accept it as a parameter. This change improves modularity and makes the function easier to test and reuse with different rates.

Task Description #2 : (Refactoring Deeply Nested Conditionals)

• Task: Use AI to refactor deeply nested if–elif–else logic into a

cleaner structure.

• Focus Areas:

o Readability

o Logical simplification

o Maintainability

Legacy Code:

score = 78

if score >= 90:

print("Excellent")

else:

if score >= 75:

print("Very Good")

else:

if score >= 60:

print("Good")

else:

print("Needs Improvement")

Expected Outcome:

o Flattened logic using guard clauses or a mapping-based approach.

Code :

```
# Refactored Code with Flattened Logic
score = 78
if score >= 90:
    print("Excellent")
elif score >= 75:
    print("Very Good")
elif score >= 60:
    print("Good")
else:
    print("Needs Improvement")
# Alternatively, using a mapping-based approach
def get_grade(score):
    if score >= 90:
        return "Excellent"
    elif score >= 75:
        return "Very Good"
    elif score >= 60:
        return "Good"
    else:
        return "Needs Improvement"
print(get_grade(score))
```

Output:

```
C:\Users\acera\Desktop\Btech_3_2\AI Assistant coding>python
Very Good
Very Good
```

Observation:

Flattening Nested Conditionals

The deep if–else logic was transformed by AI into a clean if/elif/else structure and even an alternate mapping-style get_grade function. Readability and maintainability were dramatically improved by eliminating unnecessary nesting.

Task 3 (Refactoring Repeated File Handling Code)

• Task: Use AI to refactor repeated file open/read/close logic.

• Focus Areas:

o DRY principle

o Context managers

o Function reuse

Legacy Code:

```
f = open("data1.txt")
print(f.read())
f.close()
f = open("data2.txt")
print(f.read())
f.close()
```

Expected Outcome:

o Reusable function using with open() and parameters.

Code:

```
'''
Task 3 (Refactoring Repeated File Handling Code)
• Task: Use AI to refactor repeated file open/read/close logic.
• Focus Areas:
o DRY principle
o Context managers
o Function reuse
Legacy Code:
f = open("data1.txt")
print(f.read())
f.close()
f = open("data2.txt")
print(f.read())
f.close()
Expected Outcome:
o Reusable function using with open() and parameters.
'''
# Refactored Code using Context Managers and Reusable Function
def read_file(file_name):
    with open(file_name, 'r') as f:
        return f.read()
# Example usage
print(read_file("data1.txt"))
print(read_file("data2.txt"))
```

Observation :

DRY File Handling

Repeated open/read/close sequences were consolidated into a reusable read_file helper using a context manager. AI's refactoring enforces the DRY principle and adds safety through with blocks.

Task 4 (Optimizing Search Logic)

• Task: Refactor inefficient linear searches using appropriate

data structures.

• Focus Areas:

o Time complexity

o Data structure choice

Legacy Code:

users = ["admin", "guest", "editor", "viewer"]

name = input("Enter username: ")

found = False

for u in users:

if u == name:

found = True

print("Access Granted" if found else "Access Denied")

Expected Outcome:

o Use of sets or dictionaries with complexity justification.

Code:

```
'''
Task 4 (Optimizing Search Logic)
• Task: Refactor inefficient linear searches using appropriate
data structures.
• Focus Areas:
o Time complexity
o Data structure choice
Legacy Code:
users = ["admin", "guest", "editor", "viewer"]
name = input("Enter username: ")
found = False
for u in users:
if u == name:
found = True
print("Access Granted" if found else "Access Denied")
Expected Outcome:
o Use of sets or dictionaries with complexity justification.
'''
# Refactored Code using a Set for O(1) Average Time Complexity
users = {"admin", "guest", "editor", "viewer"}  # Using a set for O(1) average time complexity
name = input("Enter username: ")
if name in users:
    print("Access Granted")
else:
    print("Access Denied")
```

Output:

```
C:\Users\acera\Desktop\Btech_3_2\AI Assistant coding>python
Enter username: admin
Access Granted
```

Observation:

Optimizing Search Logic

The linear search over a list of usernames became a constant-time membership check on a set. AI chose the appropriate data structure, explaining the O(1) average complexity advantage.

Task 5 (Refactoring Procedural Code into OOP Design)

• Task: Use AI to refactor procedural code into a class-based

design.

• Focus Areas:

o Object-Oriented principles

o Encapsulation

Legacy Code:

```
salary = 50000

tax = salary * 0.2

net = salary - tax

print(net)
```

Expected Outcome:

o A class like EmployeeSalaryCalculator with methods and

attributes

Code :

```python
'''
Task 5 (Refactoring Procedural Code into OOP Design)
• Task: Use AI to refactor procedural code into a class-based
design.
• Focus Areas:
o Object-Oriented principles
o Encapsulation
Legacy Code:
salary = 50000
tax = salary * 0.2
net = salary - tax
print(net)
Expected Outcome:
o A class like EmployeeSalaryCalculator with methods and
attributes
'''
# Refactored Code using Object-Oriented Design
class EmployeeSalaryCalculator:
    def __init__(self, salary):
        self.salary = salary

    def calculate_tax(self):
        return self.salary * 0.2

    def calculate_net_salary(self):
        tax = self.calculate_tax()
        return self.salary - tax
# Example usage
employee = EmployeeSalaryCalculator(50000)
print(employee.calculate_net_salary())
print(employee.calculate_tax())
```

Output:

```
C:\Users\acera\Desktop\Btech_3_2\AI Assistant coding>python
40000.0
10000.0
```

Observation:

Procedural → Object-Oriented

A simple salary/tax calculation was encapsulated in an EmployeeSalaryCalculator class with methods for tax and net pay. AI's rewrite adds encapsulation and adheres to OOP principles

Task 6 (Refactoring for Performance Optimization)

• Task: Use AI to refactor a performance-heavy loop handling

large data.

• Focus Areas:

o Algorithmic optimization

o Use of built-in functions

Legacy Code:

total = 0

for i in range(1, 1000000):

if i % 2 == 0:

total += i

print(total)

Expected Outcome:

o Optimized logic using mathematical formulas or

comprehensions, with time comparison.

Code:

```
'''
Task 6 (Refactoring for Performance Optimization)
• Task: Use AI to refactor a performance-heavy loop handling
large data.
• Focus Areas:
o Algorithmic optimization
o Use of built-in functions
Legacy Code:
total = 0
for i in range(1, 1000000):
if i % 2 == 0:
total += i
print(total)
Expected Outcome:
o Optimized logic using mathematical formulas or
comprehensions, with time comparison.
'''
# Refactored Code using a Mathematical Formula for the Sum of Even Numbers
n = 999998  # Largest even number less than 1,000,000
total = n * (n + 2) // 4  # Using the formula for the sum of the first k even numbers: k * (k + 1)
print(total)
```

Output:

```
C:\Users\acera\Desktop\Btech_3_2\AI Assistant coding>python
249999500000
```

Observation:

Performance Optimization

The heavy loop summing even numbers was replaced by a direct mathematical formula. AI optimized algorithmic complexity and eliminated iteration, demonstrating how built-ins and math can boost performance.

Task 7 (Removing Hidden Side Effects)

• Task: Refactor code that modifies shared mutable state.

• Focus Areas:

o Functional-style refactoring

o Predictability

Legacy Code:

```
data = []
def add_item(x):
data.append(x)
add_item(10)
add_item(20)
print(data)
```

Expected Outcome:

o Refactored function returning values instead of mutating

globals.

Code:

```
'''
Task 7 (Removing Hidden Side Effects)
• Task: Refactor code that modifies shared mutable state.
• Focus Areas:
o Functional-style refactoring
o Predictability
Legacy Code:
data = []
def add_item(x):
data.append(x)
add_item(10)
add_item(20)
print(data)
Expected Outcome:
o Refactored function returning values instead of mutating
globals.
'''
# Refactored Code using Functional-Style Refactoring
def add_item(data, x):
    return data + [x]  # Returning a new list instead of mutating the original
data = []
data = add_item(data, 10)
data = add_item(data, 20)
print(data)
```

Output:

```
C:\Users\acera\Desktop\Btech_3_2\AI Assistant coding>python
[10, 20]
```

Observation:

Removing Side Effects

A function that mutated a global list was refactored to return a new list instead. AI introduced a functional style, making behavior predictable and eliminating hidden state changes.

Task 8 (Refactoring Complex Input Validation Logic)

• Task: Use AI to simplify and modularize complex validation

rules.

• Focus Areas:

o Readability

o Testability

Legacy Code:

```
password = input("Enter password: ")
if len(password) >= 8:
if any(c.isdigit() for c in password):
if any(c.isupper() for c in password):
print("Valid Password")
else:
print("Must contain uppercase")
else:
print("Must contain digit")
else:
print("Password too short")
```

Expected Outcome:

o Separate validation functions with clear responsibility

Code:

```python
# Refactored Code with Separate Validation Functions
def is_valid_length(password):
    return len(password) >= 8

def has_digit(password):
    return any(c.isdigit() for c in password)

def has_uppercase(password):
    return any(c.isupper() for c in password)

def is_valid_password(password):
    if not is_valid_length(password):
        print("Password too short")
        return False
    if not has_digit(password):
        print("Must contain digit")
        return False
    if not has_uppercase(password):
        print("Must contain uppercase")
        return False
    print("Valid Password")
    return True
password = input("Enter password: ")
is_valid_password(password)
```

Output:

```
C:\Users\acera\Desktop\Btech_3_2\AI Assistant coding>python
Enter password: Sharan@2005
Valid Password
```

Observation:

Modular Input Validation

The convoluted password checks were broken into small validation functions (is_valid_length, has_digit, has_uppercase) and orchestrated by is_valid_password. AI's modular design enhances clarity and makes unit testing each rule straightforward.